



Développement
d'un système
embarqué pour
mesurer et
estimer la VO₂
max à partir des
tests de Léger et
Cooper.

Annexe : Code montre Cooper seul

STN « 2025 » _ « 2026 »

Projet : STN21 – Système embarqué VO₂max

Auteur: Ben said Wassim

```
#include <LilyGoLib.h>
```

```
// Include LilyGo library for board support
```

```

#include <LV_Helper.h>          // Include LVGL helper library for GUI

#define SerialGPS Serial1      // Define GPS serial port as Serial1

lv_obj_t *label;               // LVGL object pointer for main GPS data label
lv_obj_t *cooper_label;       // LVGL object pointer for Cooper test results
label
lv_obj_t *btn_start;          // LVGL object pointer for start button
lv_obj_t *btn_label;          // LVGL object pointer for button text label

// Variables for Cooper test
double prevLat = 0, prevLon = 0; // Previous latitude and longitude for
distance calculation
double distanceTotale = 0;       // Total distance traveled during test
bool firstFix = false;           // Flag indicating if first GPS fix has
been established
unsigned long startTime = 0;     // Timestamp when test started
(millisecons)
bool chronoStarted = false;      // Flag indicating if chronometer is
running
bool testPret = false;           // Flag indicating if GPS is ready for
test (valid fix)
const unsigned long DUREE_TEST = 12UL * 60UL * 1000UL; // Test duration: 12
minutes in milliseconds

// ===== SIMPLIFIED PARAMETERS =====
const int NB_POINTS_LISSAGE = 9; // Number of points for moving average
smoothing (light smoothing)
const double MIN_DIST = 5.0;     // Minimum distance threshold to
ignore GPS noise (meters)
const double MAX_DIST = 100.0;   // Maximum distance threshold to
avoid position jumps (meters)
// =====

// GPS filtering variables
static double latSum = 0, lonSum = 0; // Sum accumulators for latitude and
longitude (unused but kept)
static int count = 0;                // Counter for number of samples
(unused but kept)
double historiqueLat[NB_POINTS_LISSAGE] = {0}; // Circular buffer for
latitude history
double historiqueLon[NB_POINTS_LISSAGE] = {0}; // Circular buffer for
longitude history
int indexHistorique = 0;             // Current index in circular buffer
bool historiqueRempli = false;       // Flag indicating if circular buffer
is full

// ===== Functions =====
double toRad(double deg) {
    // Convert degrees to radians for trigonometric calculations
    return deg * 3.141592653589793 / 180.0;
}

double distanceGPS(double lat1, double lon1, double lat2, double lon2) {
    // Calculate distance between two GPS coordinates using Haversine formula
    const double R = 6371000.0; // Earth's radius in meters

```

```

double dLat = toRad(lat2 - lat1);    // Difference in latitude in radians
double dLon = toRad(lon2 - lon1);    // Difference in longitude in radians
lat1 = toRad(lat1);                  // Convert first latitude to radians
lat2 = toRad(lat2);                  // Convert second latitude to radians

// Haversine formula
double a = sin(dLat/2) * sin(dLat/2) +
           cos(lat1) * cos(lat2) *
           sin(dLon/2) * sin(dLon/2);
double c = 2 * atan2(sqrt(a), sqrt(1-a));

return R * c;                        // Return distance in meters
}

String formatTime(unsigned long millis) {
    // Convert milliseconds to formatted time string (MM:SS)
    unsigned long seconds = millis / 1000;    // Convert milliseconds to total
seconds
    unsigned long minutes = seconds / 60;      // Calculate minutes
    seconds %= 60;                            // Calculate remaining seconds

    char buffer[10];                          // Buffer for formatted string
    sprintf(buffer, "%02lu:%02lu", minutes, seconds); // Format as MM:SS with
leading zeros
    return String(buffer);                    // Return formatted string
}

// Button callback function
static void btn_start_cb(lv_event_t * e)
{
    lv_event_code_t code = lv_event_get_code(e); // Get the event code

    if (code == LV_EVENT_CLICKED) {            // Check if button was
clicked
        if (testPret && !chronoStarted) {      // Check if GPS is ready
and test not running
            chronoStarted = true;               // Start the chronometer
            startTime = millis();               // Record start time
            distanceTotale = 0;                 // Reset total distance
            firstFix = false;                   // Reset first fix flag
            prevLat = 0;                        // Reset previous latitude
            prevLon = 0;                        // Reset previous longitude
            latSum = 0;                         // Reset latitude sum
            lonSum = 0;                         // Reset longitude sum
            count = 0;                          // Reset counter
            for (int i = 0; i < NB_POINTS_LISSAGE; i++) { // Reset history
buffers
                historiqueLat[i] = 0;
                historiqueLon[i] = 0;
            }
            indexHistorique = 0;                // Reset history index
            historiqueRempli = false;           // Reset history full flag

            Serial.println(F("Cooper test started!")); // Print debug message

```

```

        // Hide button after start to prevent accidental restart
        lv_obj_add_flag(btn_start, LV_OBJ_FLAG_HIDDEN);
    }
}

void setup()
{
    Serial.begin(115200);                // Initialize serial communication
    at 115200 baud
    instance.begin();                    // Initialize LilyGo board instance
    beginLvglHelper(instance);           // Initialize LVGL helper with board
    instance

    Serial.println(F("FullExample.ino - Simplified Version")); // Print
    startup message

    // Create main GPS data label
    label = lv_label_create(lv_scr_act()); // Create label
    on active screen
    lv_obj_set_style_text_font(label, &lv_font_montserrat_20, 0); // Set font
    size 20
    lv_label_set_long_mode(label, LV_LABEL_LONG_WRAP); // Enable text
    wrapping
    lv_obj_set_width(label, 220); // Set label
    width to 220 pixels
    lv_obj_align(label, LV_ALIGN_TOP_LEFT, 5, 5); // Align to top-
    left with offset
    lv_obj_set_style_text_color(label, lv_color_make(0, 0, 0), 0); // Set text
    color to black

    // Create start button
    btn_start = lv_btn_create(lv_scr_act()); // Create button
    on active screen
    lv_obj_set_size(btn_start, 200, 50); // Set button
    size: 200x50 pixels
    lv_obj_align(btn_start, LV_ALIGN_BOTTOM_MID, 0, -10); // Align to
    bottom-center with offset
    lv_obj_add_event_cb(btn_start, btn_start_cb, LV_EVENT_CLICKED, NULL); //
    Add click event callback

    // Create button label
    btn_label = lv_label_create(btn_start); // Create label
    on button
    lv_label_set_text(btn_label, "DEMARRER TEST"); // Set button
    text to "START TEST"
    lv_obj_set_style_text_font(btn_label, &lv_font_montserrat_20, 0); // Set
    font size 20
    lv_obj_center(btn_label); // Center label
    inside button

    // Create Cooper test results label
    cooper_label = lv_label_create(lv_scr_act()); // Create label
    on active screen
    lv_obj_set_style_text_font(cooper_label, &lv_font_montserrat_20, 0); //
    Set font size 20

```

```

    lv_label_set_long_mode(cooper_label, LV_LABEL_LONG_WRAP); // Enable text
wrapping
    lv_obj_set_width(cooper_label, 220);                      // Set label
width to 220 pixels
    lv_obj_set_style_text_color(cooper_label, lv_color_make(0, 0, 0), 0); //
Set text color to black
    lv_label_set_text(cooper_label, "Test Cooper:\nEn attente..."); // Set
initial text
    lv_obj_align_to(cooper_label, label, LV_ALIGN_OUT_BOTTOM_LEFT, 0, 10); //
Position below main label

```

```

    instance.setBrightness(DEVICE_MAX_BRIGHTNESS_LEVEL);    // Set screen to
maximum brightness

```

```

    // Initialize all variables
    prevLat = 0; // Reset previous latitude
    prevLon = 0; // Reset previous longitude
    distanceTotale = 0; // Reset total distance
    firstFix = false; // Reset first fix flag
    chronoStarted = false; // Reset chronometer flag
    testPret = false; // Reset test ready flag
    latSum = lonSum = 0; // Reset sums
    count = 0; // Reset counter
}

```

```

void loop()
{
    // Read raw GPS data from the GPS module
    uint32_t satellites = instance.gps.satellites.isValid() ?
instance.gps.satellites.value() : 0; // Get number of satellites, 0 if
invalid
    double hdop = instance.gps.hdop.isValid() ? instance.gps.hdop.hdop() :
0; // Get HDOP value, 0 if invalid
    double lat = instance.gps.location.isValid() ? instance.gps.location.lat()
: 0; // Get latitude, 0 if invalid
    double lng = instance.gps.location.isValid() ? instance.gps.location.lng()
: 0; // Get longitude, 0 if invalid
    uint32_t age = instance.gps.location.isValid() ?
instance.gps.location.age() : 0; // Get GPS fix age, 0 if invalid
    uint16_t year = instance.gps.date.isValid() ? instance.gps.date.year() :
0; // Get year, 0 if invalid
    uint8_t month = instance.gps.date.isValid() ? instance.gps.date.month() :
0; // Get month, 0 if invalid
    uint8_t day = instance.gps.date.isValid() ? instance.gps.date.day() :
0; // Get day, 0 if invalid
    uint8_t hour = instance.gps.time.isValid() ? instance.gps.time.hour() :
0; // Get hour, 0 if invalid
    uint8_t minute = instance.gps.time.isValid() ? instance.gps.time.minute()
: 0; // Get minute, 0 if invalid
    uint8_t second = instance.gps.time.isValid() ? instance.gps.time.second()
: 0; // Get second, 0 if invalid
    double meters = instance.gps.altitude.isValid() ?
instance.gps.altitude.meters() : 0; // Get altitude in meters, 0 if invalid
    double kmph = instance.gps.speed.isValid() ? instance.gps.speed.kmph() :
0; // Get speed in km/h, 0 if invalid

```

```

    // Check if GPS is ready for test (valid location, at least 3 satellites,
    HDOP < 5.0)
    testPret = (instance.gps.location.isValid() && satellites >= 3 && hdop <
    5.0);

    // ===== SIMPLIFIED COOPER TEST LOGIC =====
    if (chronoStarted && instance.gps.location.isValid()) { // If test is
    running and GPS has valid location

        // Simple moving average smoothing
        historiqueLat[indexHistorique] = lat; // Store current
    latitude in history buffer
        historiqueLon[indexHistorique] = lng; // Store current
    longitude in history buffer
        indexHistorique = (indexHistorique + 1) % NB_POINTS_LISSAGE; //
    Increment index with wrap-around
        if (indexHistorique == 0) historiqueRempli = true; // Mark buffer as
    full when index wraps

        int nbPoints = historiqueRempli ? NB_POINTS_LISSAGE :
    indexHistorique; // Determine number of valid points
        double sommeLat = 0, sommeLon = 0; // Initialize
    sums for averaging
        for (int i = 0; i < nbPoints; i++) { // Loop through
    history buffer
            sommeLat += historiqueLat[i]; // Sum all
    latitudes
            sommeLon += historiqueLon[i]; // Sum all
    longitudes
        }
        double latLisse = sommeLat / nbPoints; // Calculate
    smoothed latitude
        double lonLisse = sommeLon / nbPoints; // Calculate
    smoothed longitude

        if (!firstFix) { // If this is the
    first valid position
            prevLat = latLisse; // Store as
    previous position
            prevLon = lonLisse; // Store as
    previous position
            firstFix = true; // Mark that we
    have first fix
            Serial.println("Premier point de référence établi"); // Print
    debug message
        } else { // For subsequent
    positions
            double d = distanceGPS(prevLat, prevLon, latLisse, lonLisse); //
    Calculate distance from previous point

            // Simple filter: ignore movements that are too small or too large
            if (d >= MIN_DIST && d < MAX_DIST) { // Check if
    distance is within acceptable range
                distanceTotale += d; // Add distance
    to total

```

```

        prevLat = latLisse; // Update
previous latitude
        prevLon = lonLisse; // Update
previous longitude
        Serial.printf("Distance ajoutée: +%.1f m (total: %.0f m)\n",
d, distanceTotale); // Print debug message
    }
}

// --- VO2max Calculation ---
double vo2 = 0; // Initialize
VO2max value
    if (distanceTotale > 0) { // If distance
has been recorded
        vo2 = (distanceTotale - 504.9) / 44.73; // Calculate
VO2max using Cooper test formula
    }

// --- Time tracking and test completion ---
unsigned long elapsed = chronoStarted ? (millis() - startTime) : 0; //
Calculate elapsed time in milliseconds
    String chrono = formatTime(elapsed); // Format elapsed
time as MM:SS
    String restant = chronoStarted ? formatTime(DUREE_TEST - elapsed) :
"00:00"; // Calculate remaining time
    bool testTermine = (chronoStarted && (elapsed >= DUREE_TEST)); // Check
if test is complete

// --- Display on screen ---
    lv_label_set_text_fmt(label, // Update main GPS
label with formatted data
        "Fix:%u S:%u HDOP:%.1f\n" // Fix age,
satellites, HDOP
        "Lat:%.5f\n" // Latitude
        "Lon:%.5f\n" // Longitude
        "Alt:%.0fm Vit:%.0fkm/h\n" // Altitude and
speed
        "%02d/%02d/%02d %02d:%02d", // Date and time
age, satellites, hdop,
lat, lng,
meters, kmph,
day, month, year % 100, hour, minute);

    if (!chronoStarted) { // If test is not
started
        lv_obj_clear_flag(btn_start, LV_OBJ_FLAG_HIDDEN); // Show the start
button
        if (testPret) { // If GPS is ready
            lv_obj_set_style_bg_color(btn_start, lv_color_make(0, 150, 0),
0); // Set button to green
            lv_label_set_text(cooper_label, "Test Cooper:\nGPS pret!\nAppuyez
sur DEMARRER"); // Ready message
            lv_obj_set_style_text_color(cooper_label, lv_color_make(0, 0, 0),
0); // Set text color to black

```

```

    } else { // If GPS is not
ready
        lv_obj_set_style_bg_color(btn_start, lv_color_make(100, 100, 100),
0); // Set button to gray
        lv_label_set_text(cooper_label, "Test Cooper:\nRecherche
GPS...\n(Attendez signal)"); // Waiting message
        lv_obj_set_style_text_color(cooper_label, lv_color_make(0, 0, 0),
0); // Set text color to black
    }
    } else { // If test is
running
        lv_obj_add_flag(btn_start, LV_OBJ_FLAG_HIDDEN); // Hide the start
button
        if (testTermine) { // If test is
complete
            lv_label_set_text_fmt(cooper_label, // Display final
results
                "TEST TERMINE!\n"
                "Distance: %.0f m\n"
                "VO2max: %.1f\n"
                "Temps: %s\n"
                "Sat:%u HDOP:%.1f",
                distanceTotale, vo2, chrono.c_str(), satellites, hdop);
            lv_obj_set_style_text_color(cooper_label, lv_color_make(0, 0, 0),
0); // Set text color to black
            if ((millis() / 500) % 2 == 0) { // Blink effect
every 500ms
                lv_obj_set_style_bg_color(cooper_label, lv_color_make(255, 0,
0), 0); // Set red background
                lv_obj_set_style_bg_opa(cooper_label, LV_OPA_20, 0); // Set
20% opacity
            } else {
                lv_obj_set_style_bg_opa(cooper_label, LV_OPA_0, 0); // Hide
background
            }
        } else { // If test is in
progress
            // Calculate average speed
            double vitesseMoyenne = (elapsed > 0) ? (distanceTotale / elapsed)
* 3.6 : 0; // Convert m/s to km/h

            // Build test progress text
            String texteTest = "=== TEST EN COURS ===\n";
            texteTest += " Chrono: " + chrono + "\n";
            texteTest += " Distance: " + String(distanceTotale, 0) + " m\n";
            texteTest += " VO2: " + String(vo2, 1) + "\n";
            texteTest += " Restant: " + restant + "\n";
            texteTest += " Vitesse: " + String(vitesseMoyenne, 1) + " km/h";

            lv_label_set_text(cooper_label, texteTest.c_str()); // Update
Cooper label
            lv_obj_set_style_text_color(cooper_label, lv_color_make(0, 0, 0),
0); // Set text color to black
            lv_obj_invalidate(cooper_label); // Force redraw of
label
        }
    }
}

```



```

    }

    lv_obj_align(label, LV_ALIGN_TOP_LEFT, 5, 5);          // Position main
label
    lv_obj_align_to(cooper_label, label, LV_ALIGN_OUT_BOTTOM_LEFT, 0, 10); //
Position Cooper label below main label

    // --- Serial monitor output ---
    static unsigned long lastSerialPrint = 0;              // Timer for
serial output rate limiting
    if (millis() - lastSerialPrint > 2000) {                // Print every 2
seconds
        lastSerialPrint = millis();                        // Update last
print time

        double vitesseMoyenne = (elapsed > 0) ? (distanceTotale / elapsed) *
3.6 : 0; // Calculate average speed

        Serial.printf("Sats:%2u | HDOP:%.1f | Dist:%.0fm | VO2:%.1f |
Vit.moy:%.1fkm/h | Statut:%s\r\n",
            satellites, hdop, distanceTotale, vo2, vitesseMoyenne,
            chronoStarted ? "En cours" : (testPret ? "Pret" : "Attente
GPS")); // Print status
    }

    if (testTermine) {                                     // If test is
complete
        static bool dejaAffiche = false;                  // Flag to print
summary only once
        if (!dejaAffiche) {                               // If not yet
printed
            dejaAffiche = true;                            // Mark as printed
            Serial.println(F("\n=== TEST TERMINE ===")); // Print header
            Serial.printf("Distance totale: %.2f m\n", distanceTotale); //
Print total distance
            Serial.printf("VO2max estimé: %.2f\n", vo2);   // Print estimated
VO2max
            if (vo2 < 35) Serial.println(F("VO2max: Faible")); //
Classification: poor
            else if (vo2 < 43) Serial.println(F("VO2max: Moyen")); //
Classification: average
            else if (vo2 < 52) Serial.println(F("VO2max: Bon")); //
Classification: good
            else Serial.println(F("VO2max: Excellent")); // Classification:
excellent
        }
    }

    smartDelay(500);                                       // Delay with GPS
processing
    lv_task_handler();                                     // Handle LVGL
tasks
}

static void smartDelay(unsigned long ms)
{

```

```

    // Custom delay function that processes GPS data and LVGL tasks during the
delay
    unsigned long start = millis();           // Record start
time
    unsigned long lastGPSFeed = 0;           // Timer for LVGL
task handling

    do {
        while (SerialGPS.available()) {      // Process all
available GPS serial data
            int r = SerialGPS.read();         // Read one byte
            instance.gps.encode(r);          // Feed byte to
TinyGPS++ library
        }

        if (millis() - lastGPSFeed > 20) {    // Handle LVGL
tasks every 20ms
            lv_task_handler();               // Process LVGL
tasks
            lastGPSFeed = millis();          // Update last
feed time
        }

    } while (millis() - start < ms);          // Continue until
delay time has elapsed
}

```